Collaborative Interactive Shell (CIS) — Phase 1 Proposal

Reza Kashkoul
rezakashkoul@gmail.com

February 2026

1 Project Goal

Build a collaborative terminal environment where multiple already-authenticated users on the **same machine** can attach to a single interactive shell session. Exactly one user is the **Controller** (their keystrokes go to the shell), while the rest are **Observers** (read-only output). The system must support clean attach/detach and a floor-control mechanism for requesting, granting, and revoking control.

2 Foundation Choice

2.1 Chosen approach

Build around an existing shell using a pseudo-terminal (PTY).

2.2 Exact foundation components (name + version)

- OS: Fedora Linux 43 (Workstation Edition)
- Kernel: Linux 6.17.12-300.fc43.x86_64
- Shell: `/bin/bash` GNU bash 5.3.0(1)-release

These versions are captured in `appendix/system_info.txt`.

2.3 What we get "for free" vs what we implement

Free from the foundation:

- Full interactive shell behavior (job control, pipelines, redirection, built-ins, etc.).
- Terminal semantics (line editing, prompts, control sequences) when the shell is connected to a PTY.

We must implement:

- Multi-client attach/detach.
- Real-time output broadcasting to all clients.

- Floor-control: roles (Controller/Observers), request/grant/revoke, fairness policy.
- Failure handling (controller disconnect, multiple requests, etc.).
- Logging/traces for reproducible evidence.

2.4 Terminal semantics to preserve

The implementation must preserve:

- Canonical vs raw input behavior (handled via PTY/termios configuration).
- Echoing behavior (shell/TTY echo should remain consistent for the Controller; Observers receive replicated output).
- Control sequences and interactive screen output (e.g., prompts, ANSI sequences).
- Window resizing propagation (SIGWINCH / TIOCSWINSZ) for correct terminal dimensions.

3 System Model and Architecture

3.1 High-level design

- The Host process creates a PTY and forks a child running `/bin/bash` attached to the PTY slave.
- The Host also creates a Unix domain socket (UDS) at a filesystem path (e.g., `./cis.sock`).
- Clients connect to the UDS. The Host broadcasts PTY output to all clients.
- Exactly one client is designated as Controller; only Controller input is forwarded to the PTY master.

3.2 Communication scope

This design assumes all users are already authenticated on the same machine. No additional authentication service is built. Access control is implemented using filesystem permissions on the Unix socket path.

4 Terminal Control (Floor-Control)

4.1 Roles

- **Controller**: input forwarded to the shell.
- **Observers**: output only (read-only).

4.2 Request / Grant / Revoke

Request control: Clients issue a control-request command/message. Two acceptable interfaces:

- **Out-of-band control messages** over the socket (recommended for Phase 2).
- **Simple CLI/escape commands** in the client, e.g., `/request`, `/release`.

Grant control:

- Default: FIFO queue of requests.
- Host override: Host can grant to a specific client (admin action).

Revoke control:

- Controller issues explicit `/release`.
- Host revokes/kicks a client (admin).
- Controller disconnect triggers automatic revoke and reassignment.
- Optional timeout: auto-revoke after a fixed idle time (Phase 2 option).

4.3 Fairness policy

FIFO fairness is enforced by a request queue. To avoid "spam" abuse, each client may have at most one outstanding request at a time; additional requests are ignored until the pending request is served or cancelled.

4.4 Failure handling

The design explicitly handles:

- **Controller disconnects mid-command**: Host keeps the PTY + shell running; controller role is revoked; next queued request becomes controller (or host becomes controller by default).
- **Multiple simultaneous requests**: queued FIFO; duplicates per-client are suppressed.
- **Reattach**: a disconnected user may reconnect as Observer; Controller behavior is defined by the floor-control state at reconnect time.

5 Initialization and Operating Interface

5.1 Phase 2 target CLI (spec)

```
cis host --sock ./cis.sock --shell /bin/bash
cis join --sock ./cis.sock
# in-session commands (client-side interpreted)
 /request
 /release
 /who
 /kick <user>
```

5.2 Phase 1 prototype interface (used for experiments)

For Phase 1 validation, a minimal prototype is used:

```
# host
python3 proto_host.py ./cis.sock

# clients
nc -U ./cis.sock
```

5.3 Example transcript (prototype)

```
# Terminal A (host)
$ python3 proto_host.py ./cis.sock
[CIS] host started
[CIS] socket: ./cis.sock
[CIS] shell : /bin/bash
[CIS] child pid: 512135

# Terminal B (first client becomes controller)
$ nc -U ./cis.sock
[CIS] Connected. You are CONTROLLER.
$ whoami
rkashko

# Terminal C (second client becomes observer)
$ nc -U ./cis.sock
[CIS] Connected. You are OBSERVER (read-only).
# Sees output from controller's commands
```

6 OS Mechanisms Used

- **PTYs**: `posix_openpt`, `grantpt`, `unlockpt`, `ptsname`, or a convenience call such as `forkpty`. Child uses `setsid` and `ioctl(TIOCSCTTY)` to acquire a controlling terminal.
- **Process management**: `fork/exec` to run `/bin/bash`; `waitpid` to reap; signals for cleanup.
- **I/O multiplexing**: `poll` (or `select/epoll`) on PTY master + server socket + client sockets.
- **Nonblocking I/O & backpressure**: avoid blocking on slow clients; either drop/queue bounded output for slow observers (Phase 2).
- **Terminal control**: `termios` for raw/canonical settings as needed; `ioctl(TIOCSWINSZ)` and SIGWINCH propagation for resizing.
- **Concurrency model**: single-threaded event loop (poll-based) to reduce complexity and make behavior deterministic.

7 Experiments (Reproducible)

All experiment commands are captured in `appendix/commands.txt`. Raw data and logs are in `appendix/`.

7.1 Latency + throughput (1 controller + N observers)

We measure keystroke-to-visible-echo latency using an "echo marker" method for $N \in \{1, 2, 4\}$ observers. The experiment is repeated for 50 trials per N with a per-trial timeout of 2.0s.

Command

```
python3 run_latency.py --sock ./cis.sock --Ns 1,2,4 --trials 50 --timeout 2.0 --outdir
    appendix
```

Results (from `appendix/latency.txt`)

N_{obs}	Role	Count	Min (ms)	Median (ms)	p95 (ms)	Max (ms)
1	Controller	50	0.070	0.098	0.135	0.819
1	Observers	50	0.079	0.114	0.151	0.849
2	Controller	50	0.058	0.075	0.125	1298.418
2	Observers	100	0.065	0.094	0.147	1298.474
4	Controller	50	0.126	0.139	0.164	0.528
4	Observers	200	0.137	0.186	0.224	0.597

Units & interpretation: All latency values are reported in milliseconds (ms); 1000 ms = 1 s. For $N_{obs} = 2$, one trial produced a long-tail outlier around 1298 ms (≈ 1.298 s), which makes the maximum large. We therefore treat median and p95 as the primary indicators of typical interactive latency. Raw measurements are provided in `appendix/latency_raw.csv`.

7.2 Robustness: controller disconnect mid-command

We validated that the host and shell session remain alive when the active controller disconnects mid-command. In our test, the controller issued `sleep 10; echo ROBUST_DONE`, disconnected after ~1 s, then reconnected and confirmed the session was still responsive. The output `ROBUST_DONE` appeared later after reconnect, demonstrating the command continued running despite the controller drop. Full timestamps and logs are included in `appendix/robustness.txt`.

Command

```
python3 run_robustness.py
```

Observed behavior

- Controller sends a long-ish command and disconnects abruptly.
- A new client reconnects and successfully runs `echo STILL_ALIVE`.

The full timestamped log is in `appendix/robustness.txt`.

8 Implementation Plan (Phase 2 preview)

- Replace prototype client handling with a small protocol supporting request/grant/release and participant listing.
- Enforce FIFO fairness with one-pending-request-per-client.
- Implement clean detach/reattach and robust cleanup (no orphan PTY/shell).
- Produce `demo_transcript.txt` and `POSTMORTEM.md` aligned with the proposal choices.

9 Appendix Contents (submitted separately)

- `appendix/system_info.txt`
- `appendix/commands.txt`
- `appendix/latency.txt`
- `appendix/latency_raw.csv`
- `appendix/robustness.txt`